

UNIVERSITY OF ADEN
FACULTY OF ENGINEERING
TEST OF FIRST FOR ACADEMIC YEAR (2018/2019)

Subject :Data Structure

Class :B2IT

Student Name : Ammar Tawfiq

DATE : 21/11/2018

Reg.No: 177010

Model (1)

Q1.Put (T) OR (F)

- 1- The operator used to get value at address stored in a pointer/variable is (*) .(T)
- 2- Queue – the elements are added at one end and deleted from other end (LIFO) .(F)
- 3- If the operation **pop** is performed when the stack is full, it is said to be in overflow state.(F)
- 4- A pointer - variable that stores address of an instruction. (F)
- 5- After deleting topmost element from stack, we have to decrement the value of TOP by one.(T)
- 6- The top of the stack is the index of the first element added to the stack.(F)
- 7- If Queue is full then we can insert an element into Queue.(F)
- 8- If value of FRONT variable is equal to SIZE- 1 then we can delete element from Queue.(F)
- 9- If the operation **push** is performed when the stack is full overflow state.(T)
- 10- The **Pop** operation modifies the top- the newly inserted element becomes the topmost element.(F)

Q2.What will be the output of the program ?

#Program 1

```
void main( )  
{  
    int x , y, z, *px, *py, *pz;  
    px = &x;  
    py = &y;  
    pz = &z;  
    *px = 5;  
    *py = 7;  
    *pz = *px + *py;  
    cout << *px << " " << *py << " " << *pz << endl;  
}
```

5 7 12

#Program 2

```
main( )  
{  
    int count = 22;  
    int x, *xptr;  
    xptr = &count;  
    x = *xptr;  
    cout << "count = " << count << ", x = " << x << endl;  
    return 0;  
}
```

count = 22 , x = 22

GOOD LUCK ,,,,

D.SHADA MABGER

COMPUTER ARCHITECTURE

UNIVERSITY OF AL-QADISIYA
FACULTY OF ENGINEERING
TEST OF FIRST FOR ACADEMIC YEAR (2018/2019)

Subject : Data Structure

Class : B2IT

Student Name : [REDACTED]

DATE : 21/11/2018

Reg.No. : [REDACTED]

Model (2)

Q1. Put (T) OR (F)

- 1- The operator used to get value at address stored in a pointer variable is (&) (F) ✓
- 2- Index-Sequence group of memory locations that all have the same name and type (F) ✓
- 3- Linear data structure traverses the data elements sequentially. (T) ✓
- 4- Queue - the elements are added at one end and deleted from other end (LIFO) (F) ✓
- 5- If the operation pop is performed when the stack is full, it is said to be in overflow state. (F) ✓
- 6- A pointer - variable that stores address of an instruction (T) ✓
- 7- If value of FRONT variable is equal to SIZE - 1 then we can delete element from Queue (F) ✓
- 8- If you delete an element from queue value of Rear will be decreased by 1 (F) ✓
- 9- If pop is performed when the stack is empty, then the stack is in underflow state. (T) ✓
- 10- stack is a data structure in which the elements are (popped or pushed) at only one end LIFO (T) ✓

Q2. What will be the output of the program?

#Program 1

```
int main ( )  
{  
    int count = 43;  
    int *temp;  
    int sum = 10;  
    cout << "count = " << count << " sum = " << sum << endl;  
    temp = &count;  
    *temp = 70;  
    *temp = count;  
    cout << count << " " << *temp << endl;  
    return 0;  
}
```

count = 43 sum = 10

70 70

#Program 2

```
int main ( )  
{  
    int count = 22;  
    int x, *xptr;  
    xptr = &count;  
    x = *xptr;  
    cout << "count = " << count << ", x = " << x << endl;  
    return 0;  
}
```

count = 22 x = 22

GOOD LUCK

Subject: Data Structure

Class: B2ITE

Student Name: Amirul Hossain

DATE: 21/11/2018

Reg. No: 177606

Model (1)

Q1. Put (T) OR (F)

- 1- The operator used to get value at address stored in a pointer variable is (&) (F)
- 2- Index-Sequence group of memory location that all have the same name and type (F)
- 3- Linear data structure traverses the data elements sequentially. (T)
- 4- Queue - the elements are added at one end and deleted from other end (LIFO). (F)
- 5- If the operation **pop** is performed when the stack is full, it is said to be in overflow state. (F)
- 6- A pointer - variable that stores address of an instruction (F)
- 7- If we can insert an element into stack first we have to increment the value of variable TOP by one (T)
- 8- In stacks, we cannot access data elements from any intermediate positions (T)
- 9- If Queue is empty then we cannot delete an element from Queue "Underflow". (T)
- 10- After deleting topmost element from stack, we have to decrement the value of TOP by one (T)

Q2. What will be the output of the program?

#Program 1

```
int main ( )  
{  
    int count = 20;  
    int *temp;  
    int sum = 17;  
    cout << "count = " << count << " sum = " << sum << endl;  
    temp = &count;  
    *temp = 50;  
    *temp = count;  
    cout << count << " " << *temp << endl;  
    return 0;  
}
```

count = 20 sum = 17

50 50

#Program 2

```
int main ( )  
{  
    int count = 100;  
    int x, *xptr;  
    xptr = &count;  
    x = *xptr;  
    cout << "count = " << count << ", x = " << x << endl;  
    return 0;  
}
```

count=100, x=100

UNIVERSITY OF ADEN
FACULTY OF ENGINEERING
TEST OF FIRST FOR ACADEMIC YEAR (2018/2019)

DATE: 21/11/2018

Subject : Data Structure

Class : B2ITE

Student Name :

Malek Ameen Ahmed

Reg.No: 177559

Model (2)

Q1. Put (T) OR (F)

- 1- The element that is added last will be removed first (FIFO) (F)
- 2- A pointer - variable that stores address of other variable (T)
- 3- The operator used to get value at address stored in a pointer variable (*). (T)
- 4- We can access to the value stored in the variable pointed to by using the dereferencing operator (&). (T)
- 5- If the operation *pop* is performed when the stack is full, it is said to be in overflow state. (D) (T)
- 6- If we can insert an element into stack first we have to increment the value of variable TOP by one. (T)
- 7- Linear data structure traverses the data elements sequentially (T)
- 8- In stacks, we cannot access data elements from any intermediate positions (F)
- 9- After deleting topmost element from stack, we have to decrement the value of TOP by one. (F)
- 10- If value of REAR equal to SIZE-1 then we cannot insert an element into Queue "Overflow" (F)

Q2. What will be the output of the program?

#program 1

```
int main( )
{
    int count = 88;
    int x, *xptr;
    xptr = &count;
    x = *xptr;
    cout << "count = " << count << ", x = " << x << endl;
    return 0;
}
```

Ans: count = 88

X = 88

#program 2

```
void main( )
{
    int x, y, z, *px, *py, *pz;
    px = &x;
    py = &y;
    pz = &z;
    *px = 5;
    *py = 7;
    *pz = *px + *py;
    cout << *px << " " << *py << " " << *pz << endl;
}
```

Ans: 5 7 12

GOOD LUCK ,,,

UNIVERSITY OF ADEN
FACULTY OF ENGINEERING
RE_TEST OF FIRST SEMESTER FOR ACADEMIC YEAR (2018/2019)

Subject: Data Structure

Class: B2IT / B2ITE

Student Name: Fares Mohammed Al:

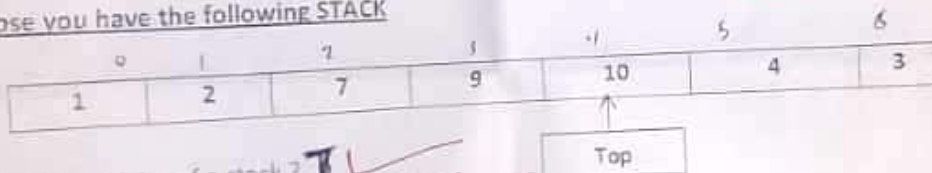
DATE: 19/12/2018

Reg. No: 173586

Q1. Put (T) OR (F)

- 1- Elements may be added to or removed from only one end, called the top of a stack. (T)
- 2- If the operation **pop** is performed when the stack is full, it is said to be in overflow state. (F)
- 3- Before every **push**, we must ensure that the stack is not empty. (F)
- 4- If value of $REAR = SIZE - 1$ then we can insert an element into Queue. (F)
- 5- The **push** operation modifies the top - the newly inserted element becomes the topmost element. (T)
- 6- If the element which is inserted in Queue is first element then we have to set the value of FRONT variable to 1. (F)
- 7- If **push** is performed when the stack is empty, then the stack - an underflow state. (F)
- 8- stack is a data structure in which the elements are (popped or pushed) at only one end - LIFO. (T)
- 9- In linear data structure, only one data element can directly be reached. (T)
- 10- If stack is not overflow then we can insert an element into stack. (T)

Q2. Suppose you have the following STACK



- 1- what is Max size of a stack? 5
- 2- what is the current size of a stack? 3
- 3- What is the last value that pushed to a stack? 10
- 4- What is the first value that pushed to a stack? 1
- 5- What is the value of the top pointer? 4

Q3. What will be the output of the program?

```
int main( )
{
    int x = 15;
    int *p = &x;
    cout << x << endl;
    cout << *p << endl;
    *p = 17;
    cout << x << endl;
    cout << *p << endl;
    x = 19;
    cout << x << endl;
    cout << *p << endl;
}
```

10